

IPD434

Redes neuronales

De evaluación de datos a arquitecturas profundas

Dr. Patricio Olivares Roncagliolo

Conexión con el curso

- Los sistemas difusos codifican conocimiento mediante membresías y reglas.
- Los algoritmos evolutivos buscan soluciones mediante poblaciones.
- Las redes neuronales ajustan parámetros a partir de ejemplos.

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n} \xrightarrow{\text{entrenamiento}} f_\theta(x)$$

> El modelo aprende regularidades de los datos. No aprende automáticamente el objetivo real ni corrige un diseño experimental deficiente.

Objetivos de la unidad

Al finalizar se espera poder:

1. **Formular** tareas de clasificación y regresión.
2. **Separar** entrenamiento, validación y prueba sin fuga de información.
3. **Seleccionar** métricas coherentes con el costo de los errores.
4. **Explicar** perceptrón, descenso de gradiente y retropropagación.
5. **Diseñar** una red multicapa con regularización.
6. **Distinguir** cuándo utilizar capas densas, recurrentes o convolucionales.

Ruta de la clase

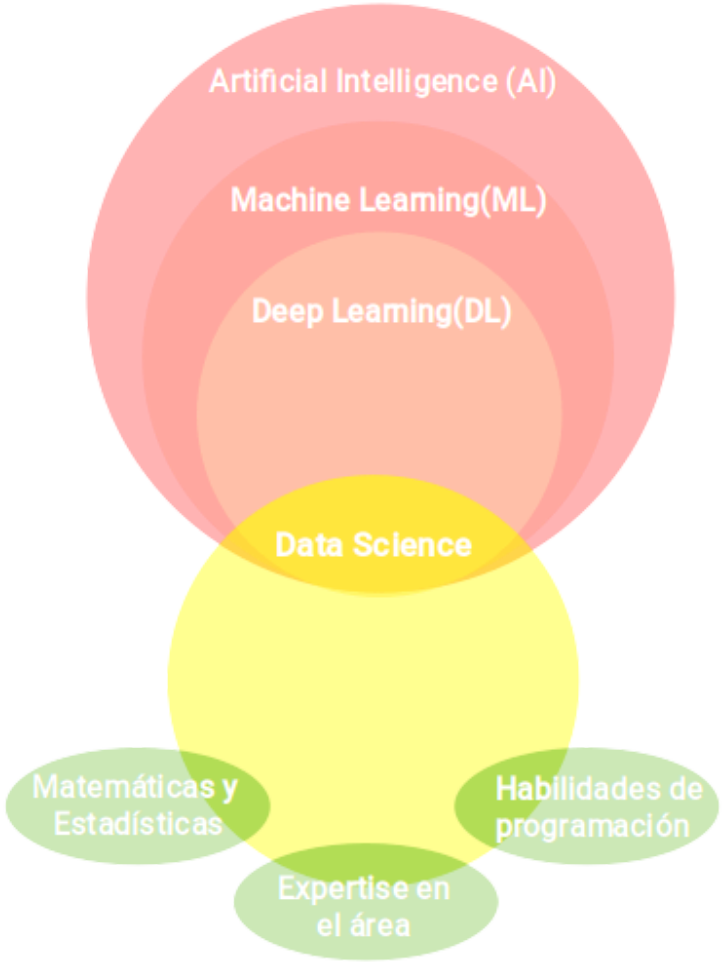
1. Definimos la tarea, los datos y el costo de equivocarse.
2. Separamos los datos y elegimos métricas antes de entrenar.
3. Construimos desde el perceptrón hasta una red multicapa.
4. Explicamos cómo descenso de gradiente y retropropagación ajustan sus parámetros.
5. Controlamos el sobreajuste y elegimos una arquitectura acorde con la estructura de los datos.
6. Cerramos con un protocolo de evaluación reproducible.

Aprendizaje Automático

El aprendizaje automático (*Machine Learning*) reúne métodos que ajustan un modelo a partir de ejemplos para producir predicciones o decisiones sobre observaciones nuevas.

> Aprender significa capturar regularidades de los datos para generalizar a observaciones que todavía no hemos visto.

Aprendizaje Automático



¿Cuándo usar aprendizaje automático?

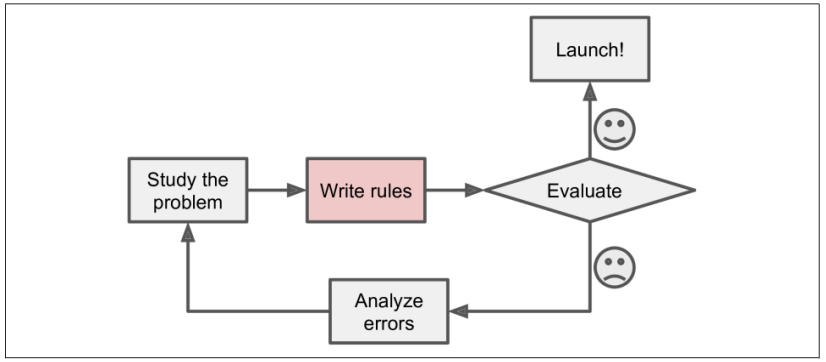


Figure 1-1. The traditional approach

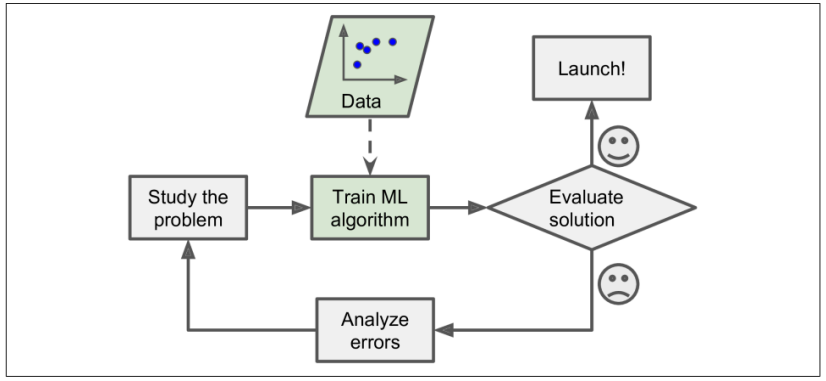


Figure 1-2. Machine Learning approach

Programación explícita

Se debe anticipar los patrones, escribir las reglas y mantener sus excepciones.

Aprendizaje desde ejemplos

Un modelo ajusta sus parámetros para capturar regularidades presentes en los datos.

Qué solución usar depende de si existe una regla simple que permita resolver el problema, o si los datos disponibles permiten obtener una relación generalizable.

Bibliotecas de Aprendizaje Automático



- **NumPy y Pandas:** representar y preparar datos.
- **Scikit-learn:** modelos clásicos, particiones y métricas.
- **PyTorch:** construir y entrenar redes neuronales.

> La biblioteca facilita el trabajo, pero la tarea, los datos y la métrica siguen siendo decisiones de diseño.

Scikit-Learn

Scikit-learn es una biblioteca de Python para aprendizaje automático. Incluye modelos de clasificación y regresión, herramientas para preparar datos y métricas para evaluar resultados.

Instalación

```
pip install -U scikit-learn
```

Enlaces

Sitio oficial

Guía de instalación

> Con la biblioteca instalada, el trabajo se organiza en objetos con responsabilidades distintas.

Scikit-Learn

Una ventaja de Scikit-Learn es la consistencia de su API: sus objetos siguen convenciones comunes y separan sus responsabilidades.

Objeto	Responsabilidad	Método típico
Estimador	Ajustar parámetros	`fit(X, y)`
Transformador	Cambiar la representación	`fit` y `transform`
Predictor	Producir salidas	`predict(X)`

Un flujo típico encadena: **ajustar con entrenamiento → transformar sin volver a ajustar → predecir sobre datos nuevos**.

Así, podemos cambiar de modelo sin rehacer toda la lógica experimental: `fit` ajusta, `transform` transforma y `predict` predice.

Tipos de Aprendizaje Automático

Los paradigmas se distinguen por la información que guía el aprendizaje.

Tipo	Información disponible	Qué busca el modelo	Ejemplo
Supervisado	Entradas con objetivo conocido	Predecir el objetivo de un caso nuevo	Estimar consumo eléctrico desde mediciones históricas
No supervisado	Datos sin objetivo	Encontrar grupos o estructura	Agrupar usuarios según su patrón de uso
Semisupervisado	Pocas etiquetas y muchos datos sin etiqueta	Aprovechar ambos conjuntos	Clasificar imágenes médicas con pocas etiquetas
Por refuerzo	Estados, acciones y recompensas	Elegir acciones con mayor retorno	Controlar el uso de energía de un sistema

Aprendizaje Supervisado

Cada ejemplo de entrenamiento contiene una entrada y su objetivo conocido:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$$

El modelo ajusta sus parámetros para que, dada una entrada nueva x, su predicción $f_\theta(x)$ se aproxime al objetivo esperado y.

> La forma del objetivo y define la tarea. Puede representar una categoría o una cantidad.

Tipos de Aprendizaje Supervisado

El objetivo y determina qué debe predecir el modelo.

Tarea	Forma del objetivo	Pregunta	Ejemplos
Clasificación	Categoría o etiqueta	¿A qué clase pertenece?	Detectar una falla, filtrar spam, identificar una prenda
Regresión	Cantidad numérica	¿Qué valor se espera?	Estimar consumo eléctrico, precio o temperatura

Esta diferencia define la salida del modelo, la función de pérdida y la métrica de evaluación.

Clasificación: predecir una categoría

Cada objetivo pertenece a una de K clases posibles:

$$f_\theta : \mathbb{R}^d \to \{1, \ldots, K\}$$

- **Clasificación binaria:** dos clases posibles, por ejemplo `falla` y `sin falla`.
- **Clasificación multiclase:** una clase entre varias posibles, como `manzana`, `plátano` o `naranja`.

> Un filtro de correo puede clasificar un mensaje como "spam" o "no spam". El objetivo es asignar una etiqueta, no medir una cantidad.

Regresión: predecir una cantidad

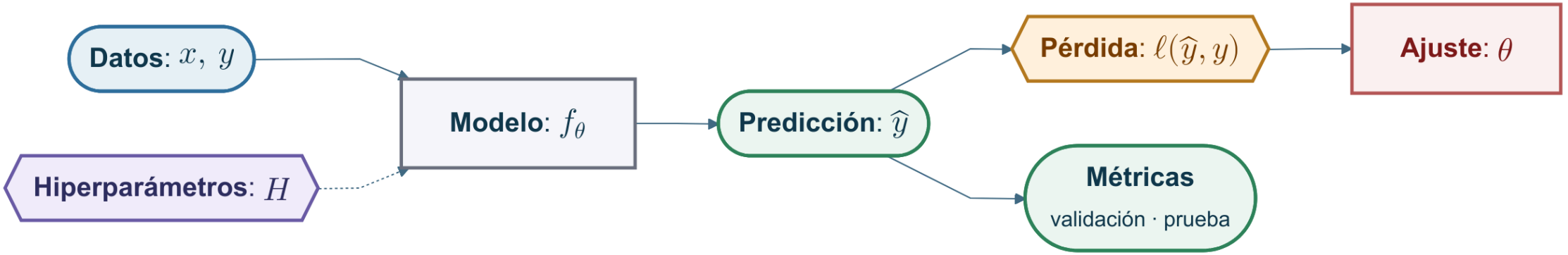
El objetivo es un valor numérico continuo, o un vector de valores:

$$f_\theta : \mathbb{R}^d \to \mathbb{R}^m$$

Por ejemplo, un modelo puede estimar el consumo eléctrico de la próxima hora en kWh, la temperatura de un equipo en $^\circ\mathrm{C}$ o el precio de una vivienda en millones.

> Un pronóstico de consumo de 120 kWh no pertenece a una clase. Debe compararse con el consumo real y el error conserva la unidad del problema.

Componentes de un experimento supervisado



> La pérdida guía el ajuste de θ con los datos de entrenamiento. La validación permite elegir hiperparámetros y la prueba se reserva para reportar el desempeño final.

Protocolo de entrenamiento y evaluación supervisada

La salida $\hat{y}$ define si la tarea es clasificación o regresión. El protocolo experimental define cómo se ajusta el modelo y cómo se estima su desempeño fuera de los datos usados para decidir.

Componente	Rol formal	Se desarrolla en
Parámetros θ y pérdida ℓ	Ajustar el modelo con entrenamiento	Pérdida promedio
Datos y preprocesamiento	Preservar la información disponible sin fuga	Particiones y preprocesamiento
Hiperparámetros $\mathcal{H}$	Elegir una configuración con validación	Selección de modelos
Métricas	Cuantificar la calidad de las predicciones	Evaluación de clasificación y regresión

Cómo se ajustan los parámetros del modelo

Sea $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^{n_{\text{train}}}$. Cada ejemplo produce un error mediante la pérdida $\ell(\hat{y}, y)$

.

$$\widehat{L}_{\text{train}}(\theta) = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \ell\big(f_\theta(x_i), y_i\big)$$

El entrenamiento resume los errores individuales en una pérdida promedio y busca los parámetros que la minimizan:

$$\hat{\theta} = \arg\min_{\theta} \widehat{L}_{\text{train}}(\theta)$$

Esta optimización responde cuánto se equivoca el modelo en los ejemplos disponibles para aprender. Un error bajo allí no garantiza un buen desempeño en ejemplos nuevos.

Parámetros e hiperparámetros en el aprendizaje

Los **parámetros** son valores internos que el modelo aprende, como pesos y sesgos.
Los **hiperparámetros** configuran el modelo y su entrenamiento, como arquitectura, tasa de aprendizaje, tamaño de lote y regularización.

$\mathcal{A}_{\mathbf{h}}$ es el procedimiento de entrenamiento definido por $\mathbf{h}$. Usa $\mathcal{D}_{\text{train}}$ para calcular gradientes y actualizar los parámetros.

$$\hat{\theta}(\mathbf{h}) = \mathcal{A}_{\mathbf{h}}\left(\mathcal{D}_{\text{train}}\right)$$

$\hat{\theta}(\mathbf{h})$ son los parámetros finales aprendidos con esa configuración.

Selección de hiperparámetros mediante validación

Cada configuración $\mathbf{h}$ produce una función entrenada $f_{\hat{\theta}(\mathbf{h})}$, definida por los parámetros obtenidos. La validación permite comparar esas funciones sin usar todavía el conjunto de prueba:

$$\mathbf{h}^* = \arg\min_{\mathbf{h}\in\mathcal{H}} \widehat{L}_{\text{val}}\left(\hat{\theta}(\mathbf{h})\right)$$

- $\mathcal{H}$: conjunto de configuraciones candidatas.
- $\widehat{L}_{\text{val}}$: pérdida promedio calculada sobre validación.
- $\mathbf{h}^*$: configuración cuyo modelo obtiene la menor pérdida de validación.

La prueba se reserva para estimar el desempeño final de la configuración seleccionada.

Particiones para ajuste, selección y evaluación

El conjunto de datos se divide en subconjuntos disjuntos:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \,\dot\cup\, \mathcal{D}_{\text{val}} \,\dot\cup\, \mathcal{D}_{\text{test}}$$

Partición	Uso	¿Ajusta decisiones?
Entrenamiento	Estimar parámetros	Sí
Validación	Elegir hiperparámetros y detener	Sí
Prueba	Estimar desempeño final	No

> Preprocesar con estadísticas calculadas sobre todos los datos, seleccionar modelos mirando prueba o duplicar muestras entre particiones produce fuga de información.

Porcentajes habituales de partición

Los porcentajes son orientativos. La proporción se expresa como entrenamiento / validación / prueba.

Estructura	Proporción	Cuándo usarla
Entrenamiento / prueba	80 / 20	Caso general, con modelo definido o validación cruzada
Entrenamiento / prueba	70 / 30 o 60 / 40	Dataset grande y necesidad de una prueba más estable
Entrenamiento / validación / prueba	80 / 10 / 10	Búsqueda moderada de hiperparámetros
Entrenamiento / validación / prueba	70 / 15 / 15	Dataset grande o comparación amplia de configuraciones
Dataset muy grande	90+ / 5 / 5	Muchas observaciones incluso en validación y prueba

Preprocesamiento ajustado solo con entrenamiento

Un transformador, por ejemplo un escalador o codificador, debe estimarse únicamente en $\mathcal{D}_{\text{train}}$:

$$\widehat{T} = \text{fit}(X_{\text{train}}), \qquad X'_s = \widehat{T}(X_s), \quad s \in \{\text{train}, \text{val}, \text{test}\}$$

Esto permite aplicar la misma representación a cada partición sin incorporar información de validación o prueba durante el ajuste.

Secuencia de selección y prueba final

1. **Dividir** los datos respetando grupos, tiempo y clases cuando corresponda.
2. **Ajustar** el preprocesamiento solo con entrenamiento.
3. **Transformar** entrenamiento, validación y prueba con esos mismos parámetros.
4. **Seleccionar** arquitectura e hiperparámetros usando validación.
5. **Evaluar** la prueba una sola vez al final.

Métricas de evaluación

Una métrica resume la calidad de las predicciones comparando los objetivos reales y con las predicciones $\hat{y}$:

$$M\left(\mathbf{y}, \hat{\mathbf{y}}\right) \in \mathbb{R}$$

La métrica debe responder a la decisión que se quiere apoyar. Por eso importa:

- qué error se considera relevante
- si las clases están desbalanceadas
- qué unidades tiene el objetivo
- qué costo tiene equivocarse

Una misma predicción puede parecer buena o mala según la métrica utilizada.

Métricas para clasificación

En clasificación, la predicción es una etiqueta o un puntaje que luego se convierte en etiqueta mediante un umbral.

En clasificación binaria, la calidad del modelo se describe contando aciertos y errores: verdaderos positivos (TP), verdaderos negativos (TN), falsos positivos (FP) y falsos negativos (FN).

Clasificación binaria: aciertos y errores

Supongamos que la clase positiva representa la condición que queremos detectar:

Resultado	Condición real	Predicción	Interpretación	Ejemplo
TP	Positiva	Positiva	Acierto al detectar la condición	 esto es 5
TN	Negativa	Negativa	Acierto al descartar la condición	 esto no es 5
FP	Negativa	Positiva	Falsa alarma	 esto no es 5
FN	Positiva	Negativa	Omisión de un caso real	 esto es 5

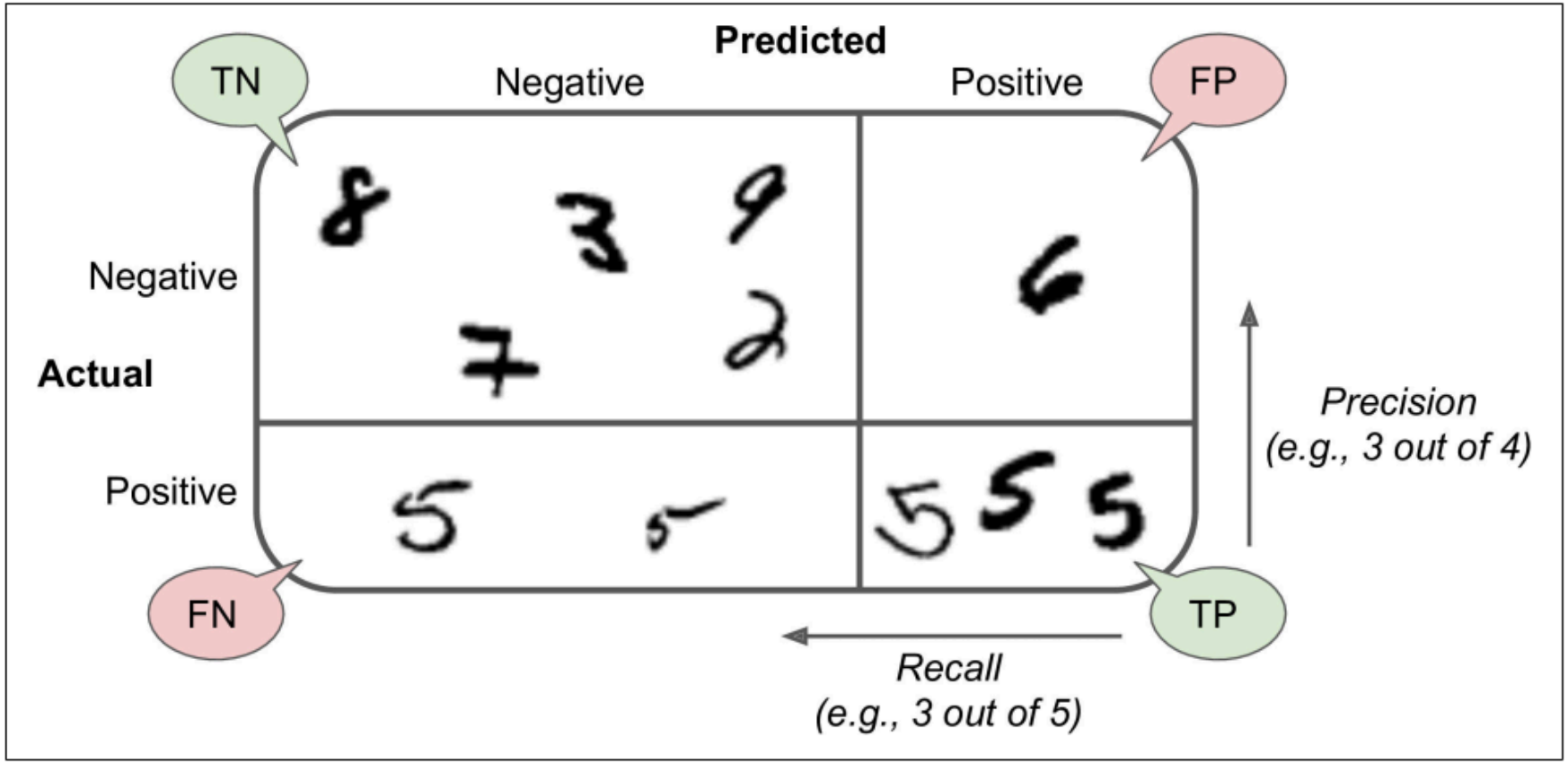
El significado de "positivo" depende de la decisión. No tiene por qué corresponder a la clase más frecuente.

Matriz de confusión binaria

Una matriz de confusión cruza la clase real con la clase predicha y cuenta cuántos casos caen en cada combinación.

En clasificación binaria, sus cuatro celdas corresponden a TP, TN, FP y FN. Así permite observar por separado los aciertos y los tipos de error.

Matriz de confusión binaria



Cómo leer una matriz de confusión

La matriz cruza dos perspectivas sobre los mismos casos:

- **Filas:** condición real observada.
- **Columnas:** clase predicha por el modelo.
- **Diagonal:** aciertos TP y TN.
- **Fuera de la diagonal:** errores FP y FN.

La matriz conserva los conteos necesarios para calcular exactitud, precisión, exhaustividad y especificidad. La importancia de cada celda depende del problema.

Cálculo de métricas a partir de una matriz de confusión

El notebook 04_clasificacion_titanic.ipynb reproduce el cálculo sobre el dataset Titanic: Machine Learning from Disaster. Sobre sus 179 casos de prueba:

	Predice positivo	**Predice negativo**
Real positivo	$TP = 59$	$FN = 17$
Real negativo	$FP = 16$	$TN = 87$

- Exactitud: $(59 + 87)/179 = 81{,}6\%$.
- Precisión: $59/(59 + 16) = 78{,}7\%$.
- Exhaustividad: $59/(59 + 17) = 77{,}6\%$.

La misma matriz puede verse "bien" o "mal" según qué error sea más costoso.

Exactitud

Exactitud (*accuracy*)

Aciertos sobre el total de casos:

$$\text{Acc} = \frac{TP + TN}{TP + TN + FP + FN}$$

- ¿Qué proporción del total fue clasificada correctamente?
- Usar cuando las clases y los costos de error son similares.

Especificidad

Especificidad (*specificity*, TNR)

Negativos reales reconocidos correctamente:

$$\text{Esp} = \frac{TN}{TN + FP}$$

- ¿Qué proporción de los negativos reales fue descartada correctamente?
- Usar cuando importa no marcar como positivos los negativos reales y una falsa alarma FP resulta costosa.

Precisión

Precisión (*precision*, PPV)

Positivos reales entre las predicciones positivas:

$$\text{Prec} = \frac{TP}{TP + FP}$$

- ¿Cuántos casos marcados como positivos eran realmente positivos?
- Usar cuando una falsa alarma FP es costosa en relación con omitir positivos.

Exhaustividad

Exhaustividad (*recall*, sensibilidad, TPR)

Positivos reales detectados por el modelo:

$$\text{Rec} = \frac{TP}{TP + FN}$$

- ¿Cuántos positivos reales fueron detectados?
- Usar cuando una omisión FN es costosa en relación con generar falsas alarmas.

Limitaciones de las métricas

Consideremos 20 números: 3 son 5 y 17 no son 5.

El modelo "nada es 5" predice "no es 5" para todos los casos:

$$TP = 0, \qquad TN = 17, \qquad FP = 0, \qquad FN = 3$$

Su exactitud es:

$$\text{Acc} = \frac{0 + 17}{20} = 85\%$$

El valor parece alto, pero el modelo no detecta ninguno de los tres cincos.

Limitaciones de las métricas

Para el modelo "nada es 5", las otras métricas muestran un comportamiento muy diferente:

$$\text{Rec} = \frac{0}{0+3} = 0, \qquad \text{Esp} = \frac{17}{17+0} = 1$$

La precisión es indefinida porque el modelo no predice ningún positivo. Una biblioteca puede aplicar una convención configurable para evitar esta división por cero:

$$\text{Prec} = \frac{0}{0+0} = \frac{0}{0}$$

La exactitud está dominada por la clase mayoritaria. Por eso debemos examinar qué tipo de acierto o error importa en el problema.

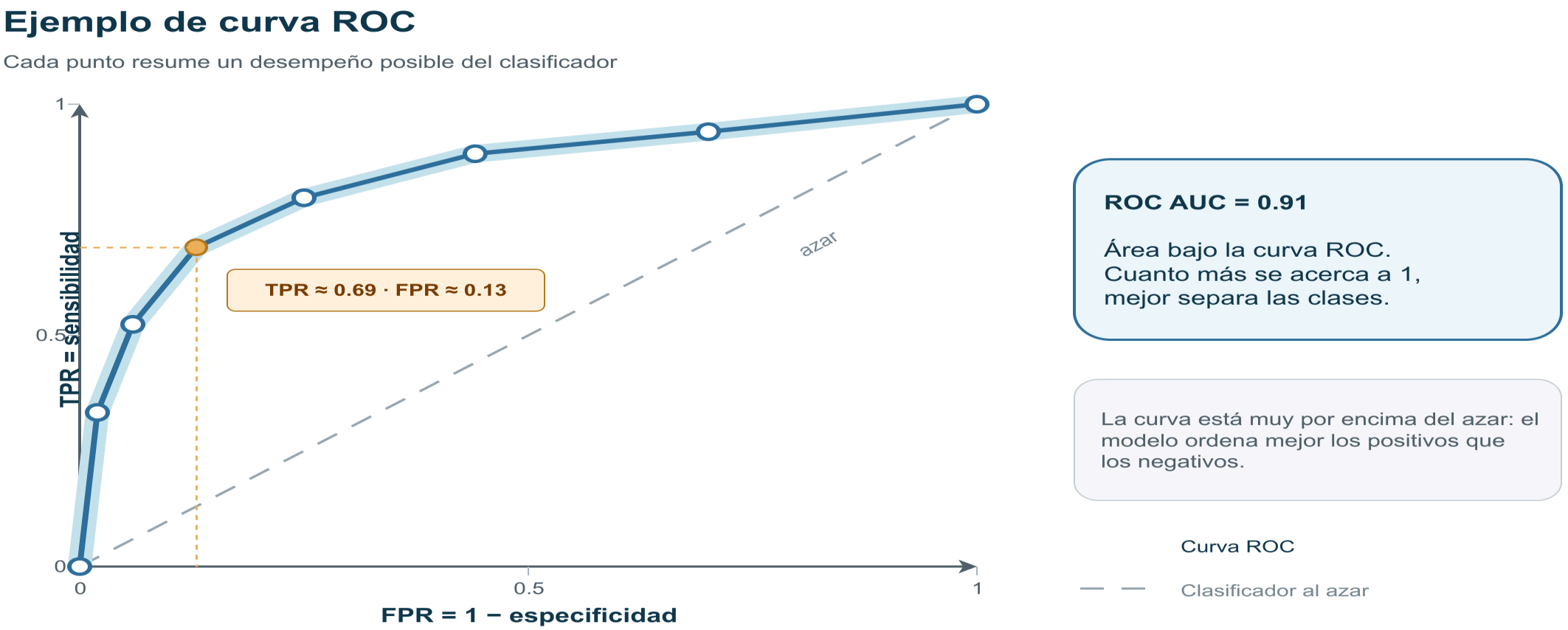
F_1: equilibrio entre precisión y exhaustividad

$$F_1 = 2\frac{\mathrm{Prec} \cdot \mathrm{Rec}}{\mathrm{Prec} + \mathrm{Rec}}$$

El F_1 es la media armónica de precisión y exhaustividad.

- Solo es alto cuando ambas métricas son altas.
- Es útil cuando precisión y exhaustividad tienen una importancia comparable.

Curva ROC y ROC AUC



La curva ROC muestra la sensibilidad frente a la tasa de falsos positivos. ROC AUC resume la capacidad del modelo para separar ambas clases.

Métricas para regresión

En regresión, la predicción es una **cantidad numérica**. Las métricas comparan y y $\hat{y}$ mediante una noción de distancia o variación explicada.

MAE: Mean Absolute Error

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

El MAE es el error absoluto promedio. Cada unidad de error contribuye de la misma forma, sin importar si la predicción quedó por encima o por debajo del valor real.

- Usarlo cuando se necesita una medida directa e interpretable en las unidades del objetivo.
- Es menos sensible a valores extremos que las métricas cuadráticas.

MSE: Mean Square Error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

El MSE eleva cada error al cuadrado. Por eso un error grande pesa mucho más que varios errores pequeños.

- Usarlo cuando los errores grandes son especialmente costosos o deben penalizarse con fuerza.
- Su unidad queda elevada al cuadrado y es sensible a valores extremos.

RMSE: Root Mean Square Error

$$RMSE = \sqrt{MSE}$$

El RMSE conserva las unidades de la variable objetivo y resume la magnitud típica del error, dando más peso a las desviaciones grandes.

- Usarlo cuando se quiere relacionar el desempeño con una tolerancia operacional.
- Mantiene la sensibilidad del MSE frente a valores extremos.

R^2: comparación con una línea base

El coeficiente de determinación compara el error cuadrático del modelo con el de predecir siempre la media del conjunto evaluado:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

$R^2 = 1$ indica ajuste perfecto. $R^2 = 0$ corresponde al predictor constante $\bar{y}$ en ese conjunto evaluado. Un valor negativo indica que el modelo rinde peor que esa línea base.

- Usarlo para comparar cuánto mejora el modelo respecto de una referencia simple.
- Complementarlo con MAE o RMSE para conocer el error en las unidades originales.

Modelos predictivos supervisados

Un modelo predictivo aprende una función parametrizada a partir de datos etiquetados:

$$f_\theta : \mathcal{X} \longrightarrow \mathcal{Y}, \qquad \hat{y} = f_\theta(\mathbf{x})$$

Los parámetros θ se ajustan con entrenamiento y luego se evalúan sobre datos no vistos.

- **Clasificación:** $\mathcal{Y}$ contiene etiquetas discretas.
- **Regresión:** $\mathcal{Y} \subseteq \mathbb{R}^m$ contiene valores numéricos.

La tarea fija el tipo de salida, la función de pérdida y las métricas de evaluación.

Ejemplos de modelos predictivos en Scikit-Learn

Clasificación

- Regresión logística: `LogisticRegression`
- KNN: `KNeighborsClassifier`
- Árboles y bosques: `DecisionTreeClassifier`, `RandomForestClassifier`
- Support Vector Machine Classifier: `SVC`

Regresión

- Modelos lineales: `LinearRegression`, `Ridge`, `Lasso`
- KNN: `KNeighborsRegressor`
- Árboles y ensambles: `RandomForestRegressor`, `GradientBoostingRegressor`
- Support Vector Machine Regressor: `SVR`

Modelo de regresión lineal

Consideremos m ejemplos de entrenamiento y N características por ejemplo. Para incorporar el intercepto, definimos $x_0^{(i)} = 1$:

$$h_\theta\left(x^{(i)}\right) = \sum_{j=0}^{N} \theta_j x_j^{(i)} = \theta_0 + \sum_{j=1}^{N} \theta_j x_j^{(i)}$$

- θ_0 es el intercepto y θ_j es el peso de la característica j.
- $i = 1, \ldots, m$ identifica el ejemplo y $j = 1, \ldots, N$ la característica.

El modelo es lineal en los parámetros, aunque las características pueden provenir de transformaciones no lineales.

Mínimos cuadrados ordinarios

El entrenamiento busca los valores de $\theta_0, \ldots, \theta_N$ que minimizan el error cuadrático promedio:

$$\mathrm{MSE}(\theta) = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right)^2$$

$$\hat{\theta}_0, \ldots, \hat{\theta}_N = \underset{\theta_0, \ldots, \theta_N}{\operatorname{argmin}} \ \mathrm{MSE}(\theta)$$

Cada residuo $h_\theta\left(x^{(i)}\right) - y^{(i)}$ mide el error de un ejemplo. Elevarlo al cuadrado penaliza más las desviaciones grandes.

Condición de óptimo en mínimos cuadrados

La derivada parcial respecto de cada parámetro es:

$$\frac{\partial \operatorname{MSE}(\theta)}{\partial \theta_j} = \frac{2}{m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right) x_j^{(i)}, \qquad j = 0, \ldots, N$$

En el mínimo, todas las derivadas se anulan. Por tanto, los parámetros satisfacen las ecuaciones normales escalares:

$$\sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right) x_j^{(i)} = 0, \qquad j = 0, \ldots, N$$

Estas ecuaciones forman un sistema de $N + 1$ ecuaciones para $\theta_0, \ldots, \theta_N$. Resolverlo entrega los parámetros que minimizan el MSE.

Descenso de gradiente en regresión lineal

También podemos ajustar cada parámetro mediante actualizaciones iterativas:

$$\theta_j \leftarrow \theta_j - \eta \frac{\partial \, \mathrm{MSE}(\theta)}{\partial \theta_j}, \qquad j = 0, \ldots, N$$

- $\eta > 0$ es la tasa de aprendizaje.
- El gradiente indica cómo cambia el MSE al modificar cada parámetro.

La misma regla se extenderá a redes neuronales, donde no existe en general una solución cerrada.

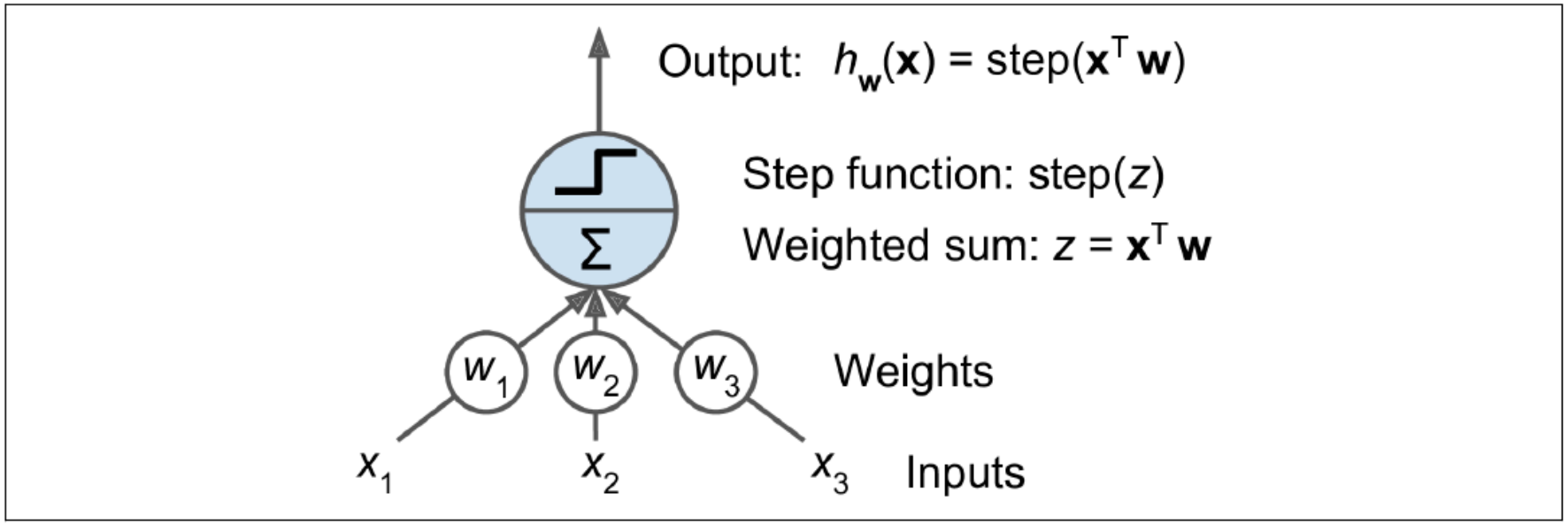
Redes Neuronales

Las redes neuronales son una familia de modelos paramétricos ampliamente utilizada para aprender relaciones complejas y generar nuevas salidas a partir de patrones aprendidos.

- **Predicción:** clasificación de imágenes, recomendación, pronóstico y detección de anomalías.
- **Modelos generativos:** creación de texto, imágenes, audio, video y código.
- **LLM:** modelos de lenguaje a gran escala que permiten asistentes como ChatGPT, Claude y otros sistemas conversacionales.

> La misma idea de apilar unidades simples permite construir modelos predictivos y generativos.

Redes Neuronales: El Perceptrón



Redes Neuronales: El Perceptrón

$$z = w_0 + \sum_{j=1}^{N} w_j x_j, \qquad \hat{y} = \text{step}(z)$$

La función escalón convierte el puntaje en una etiqueta binaria:

$$\text{step}(z) = \begin{cases} 0, & z < 0 \\ 1, & z \geq 0 \end{cases}$$

Los pesos definen la orientación de la frontera y w_0 la desplaza. Una sola unidad solo puede separar clases linealmente separables.